

Interview Prep Answers

Ekaggata – Société Générale · DevOps Interview Questions

1. Linux / Shell

Self-rate: be honest based on real usage. Given Terraform/Ansible/Docker work, 3.5–4/5 is defensible — justify with file permissions, process management, grep/awk/sed, systemd, cron.

Difference between `./script.sh` and `sh script.sh`

• **`./script.sh`** — runs the script as an executable using the interpreter in its shebang (`#!/bin/bash`). Requires execute permission (`chmod +x`).

• **`sh script.sh`** — explicitly invokes `sh` to interpret the script, ignoring the shebang and execute permission entirely. Runs even without `+x`, but behaviour may differ if the script uses bash-specific syntax and `sh` is symlinked to `dash`.

How to run a script without giving explicit execute permission

Pass it as an argument to the interpreter rather than executing it directly: **`bash script.sh`** or **`sh script.sh`**.

2. Scheduling / Automation

If you haven't used enterprise schedulers, say so honestly and pivot to what you have used: **`cron`** (Linux crontab), GitHub Actions scheduled workflows (on: schedule with cron syntax), and Kubernetes CronJobs. Don't claim Control-M/Autosys if you haven't touched them — this is a bank interview, they will drill down.

3. Production / Database Troubleshooting

Stuck session blocking a chain of executions — what would you tell the DBA?

Identify the blocking session (`pg_stat_activity` in Postgres, `v$session/v$lock` in Oracle), find the SPID/PID holding the lock, check what query it's running and how long it's been active. If genuinely hung (not just slow), kill the session (`pg_terminate_backend()` / `ALTER SYSTEM KILL SESSION` in Oracle) — but that decision belongs to the DBA, not the DevOps engineer. Your role: surface the blocking chain with session IDs, wait events, and lock type, and recommend killing the blocker or waiting out a timeout depending on transaction criticality.

If you haven't personally resolved this live, say so — note you understand it conceptually via `pg_locks/pg_stat_activity` but haven't owned the resolution.

4. Terraform

• **Static infra already exists — why Terraform?** Drift detection, repeatability, disaster recovery, version-controlled infra history, avoiding manual console changes going forward.

• **Starting Terraform on existing manual infra:** `terraform import` to bring existing resources into state, write matching HCL, reconcile with `terraform plan` until diff is zero.

• **Value on already-stable infra:** future changes become auditable/reviewable via PR, rollback becomes possible, team scaling is safer (no tribal knowledge in one person's head).

- **Running apply again with no changes:** no recreation. Terraform compares desired state (HCL) vs current state (statefile) vs real infra; no diff means no-op.
 - **Changing RAM/storage on an existing instance, same name:** yes — modify the `instance_type/size` attributes in HCL and apply. If the change is in-place-updatable, Terraform updates the same resource (same ID); if it requires replacement, Terraform destroys and recreates — check “Update requires replacement” in the plan output.
 - **Folder structure:** answer from your actual project — `main.tf`, `variables.tf`, `outputs.tf`, `provider.tf`, `terraform.tfvars`, `modules/` directory, remote backend (S3 + DynamoDB).
-

5. Kubernetes / EKS

Answer from your actual hands-on work: minikube Deployments/StatefulSets, and if your capstone uses EKS, walk through that architecture — ingress, services, pod-to-pod communication, ConfigMaps/Secrets, how the app scales. Describe your own project's topology, don't generalize.

6. Argo CD / GitOps

Argo CD uses **polling** by default (configurable interval, default ~3 min) to detect drift in the Git repo — not native hooks, though webhooks can be configured to trigger an immediate refresh instead of waiting for the poll interval.

Key architectural distinction (this is what the interviewer was testing): Argo CD does not decide new image tags or rewrite the manifest itself in that sense. The CI pipeline (Jenkins/GitHub Actions) builds the new image, then updates the manifest/Kustomize overlay/Helm values file in Git (commit), either directly or via a tool like `argocd-image-updater`. Argo CD's job is purely to **detect** that the Git state changed and **reconcile** the cluster to match — a pull-based GitOps controller, not a CI tool. Jenkins owns “what changes,” Argo CD owns “make the cluster match Git.”

7. Docker / Image / Layer Troubleshooting

- **Image/ConfigMap conflict on deploy:** check `kubectl describe pod` events, `kubectl logs`, verify the image tag matches intent (not a stale cached tag), confirm the ConfigMap is actually mounted/synced — a ConfigMap change doesn't auto-restart pods unless you use a checksum annotation or a tool like Reloader.
- **Single image, internal component conflict:** inspect layers with `docker history <image>`, check for version mismatches in the final layer, rebuild with `--no-cache` to rule out stale cache.
- **On rebuild:** Docker only rebuilds layers from the first changed instruction onward (layer caching); everything above that Dockerfile line is reused from cache.
- **Does Docker remove/replace layers?** No — image layers are immutable. A “changed” layer is a new layer that overlays/shadows the old one in the union filesystem (OverlayFS), not an in-place edit. Deleted files show up as whiteout markers in the new layer rather than physical removal from earlier layers — which is why layer history persists and image size can balloon from files deleted in earlier layers.
- **Cause of conflict between two layers:** typically a file path collision where a later layer's file shadows/overwrites an earlier layer's file at the same path — not a git-merge-style conflict, the topmost layer simply wins.

- **Same path changed in another layer:** the topmost layer's version is what's visible at runtime; the earlier layer's version still exists in image history/size but isn't visible.
 - **Is layering idempotent?** Building the same Dockerfile with the same context reproducibly produces the same layer contents/hashes, assuming deterministic base images and no timestamp/cache-busting steps — though real-world non-determinism (apt updates, :latest tags) can break this in practice.
-

8. Kubernetes Troubleshooting — CrashLoopBackOff

- `kubectl describe pod <pod>` — check events, exit code, last state reason.
- `kubectl logs <pod> --previous` — see why the last instance died.
- Check resource limits — OOMKilled shows as exit code 137.
- Check liveness/readiness probe config — failing probes cause restarts.
- Verify the container's entrypoint/command is correct.
- Check ConfigMap/Secret references exist and are correctly mounted.
- Common recurring causes: bad image tag, missing env vars/secrets, app crashing on startup due to a failed dependency (DB not reachable), misconfigured probes with too-short timeouts, and OOM from under-provisioned memory limits.